

Valid Parentheses (LeetCode 20)

Algorithm Notes & Implementation

Authored by: Arpit Raj, LNMIIT Jaipur

Date: September 9, 2026

Topic: Stack Data Structure / String Parsing

Logic

Visual Representation & Overview

The solution relies on a **Stack (LIFO)** to track unclosed brackets sequentially:

- Whenever we encounter an opening bracket (`(` , `[` , or `{`), we push it onto the stack to await its matching pair.
- When encountering a closing bracket (`)` , `]` , or `}`), we expect it to match the most recently seen unclosed bracket (which sits at the top of the stack).
- **Invalid Conditions:**
 - If we see a closing bracket but the stack is empty (nothing to close).
 - If the closing bracket doesn't match the top of the stack.
 - If, at the end of the string, the stack isn't empty (unmatched opening brackets remain).
- If the loops completes and the stack is perfectly empty, the string is valid.

```
class Solution {
public:
    bool isValid(string s) {

        stack<char> st;

        for(char c : s){
            if(c == '(' || c == '[' || c == '{'){
                st.push(c);
            }
            else{
                if(st.empty()){
                    return false;
                }
                if(c == ')' && st.top() != '(') return false;
                else if(c == '}' && st.top() != '{') return false;
                else if(c == ']' && st.top() != '[') return false;

                st.pop();
            }
        }

        if(st.empty()) return true;
        else return false;
    }
};
```